Electrify

EV Charging Station Management and Booking Platform

A Full-Stack Electric Vehicle Charging Station Management System for Booking, Managing, and Monitoring EV Charging Infrastructure

Team Members

Kavya Katal

Ankit Kumar Pandey

Yashi Agrawal

Kavya Mukhija

Khushi Batra

SD Group Project

April 27, 2026

Contents

1 Introduction **3**
- 1.1 Project Overview . . . 3
- 1.2 Objectives . . . 3
- 1.3 Scope . . . 3

2 Project Motivation **3**

3 Problem Statement **4**

4 Proposed Solution **4**

5 System Architecture **4**
- 5.1 Architectural Style . . . 4
  - 5.1.1 Backend Layers . . . 4
  - 5.1.2 Frontend Layers . . . 5
- 5.2 Technology Stack . . . 5
- 5.3 Project Directory Structure . . . 5

6 Core Features **6**
- 6.1 User Personas and Role-Based Access . . . 6
- 6.2 Authentication and Security . . . 6
- 6.3 Station and Machine Management . . . 6
- 6.4 Booking Lifecycle . . . 7
- 6.5 Payments . . . 7
- 6.6 Notifications . . . 7

7 UML Diagrams **7**
- 7.1 Use Case Diagram . . . 8
  - 7.1.1 Customer Use Cases . . . 8
  - 7.1.2 Manager Use Cases . . . 9
  - 7.1.3 Admin (Station Owner) Use Cases . . . 9
  - 7.1.4 SuperAdmin Use Cases . . . 9
- 7.2 Sequence Diagram . . . 10
  - 7.2.1 Flow 1: Customer Registration and Slot Booking . . . 10
  - 7.2.2 Flow 2: Station Registration and Admin Approval . . . 11
- 7.3 UML Class Diagram . . . 12
  - 7.3.1 Core Entities . . . 12
  - 7.3.2 Supporting Entities . . . 12
  - 7.3.3 Enumerations . . . 13
- 7.4 Entity–Relationship Diagram . . . 14
  - 7.4.1 Tables and Key Relationships . . . 15

8 System Design Patterns **15**
- 8.1 Singleton Pattern . . . 15
- 8.2 Factory Pattern . . . 16
- 8.3 Repository Pattern . . . 16
- 8.4 Layered Architecture Pattern . . . 16

9 SOLID Principles **17**
9.1 Single Responsibility Principle (SRP) 17
9.2 Open/Closed Principle (OCP) 17
9.3 Liskov Substitution Principle (LSP) 17
9.4 Interface Segregation Principle (ISP) 17
9.5 Dependency Inversion Principle (DIP) 17

10 API Modules **17**
10.1 Authentication & Account Management — `/api/auth` 18
10.2 User Profile Management — `/api/user` 18
10.3 Station Requests — `/api/station-request` 18
10.4 Charging Stations — `/api/station` 19
10.5 Health & System — `/api/health` 19
10.6 Authorization Notes 19

11 Frontend UI Design **20**
11.1 Pages 20
11.2 Key Components 20
11.3 Application Layer 20

12 Testing **20**

13 Challenges Faced **21**

14 Future Scope **21**

15 Environment Configuration **21**
15.1 Required Environment Variables 22
15.2 Optional Environment Variables 22

16 Setup and Deployment **22**
16.1 Prerequisites 22
16.2 Local Development Setup 22
16.3 Available Scripts 23

17 Conclusion **23**

1 Introduction

1.1 Project Overview

Electrify is a full-stack Electric Vehicle charging station management and booking platform. The project is designed to solve the growing problem of EV charging accessibility by connecting customers, station owners, managers, and superadmins through a single digital system.

The platform allows customers to find charging stations, book slots, manage their profile, and track charging activity. Station owners can apply for station registration, manage machines, assign managers, and monitor station operations. Superadmins can review station owner requests, verify stations, and control platform-level approvals.

1.2 Objectives

- Provide a digital EV charging station booking system.
- Allow station owners to register and manage charging stations.
- Support multiple user roles with separate dashboards.
- Prevent booking conflicts on charging machines.
- Provide secure login, signup, and profile management.
- Support document/image upload for verification using Cloudinary.
- Build a scalable backend using clean architecture principles.
- Create a modern and responsive frontend using React and Tailwind CSS.

1.3 Scope

The system covers the following functional domains:

1. **User Management:** Registration, authentication, role assignment, and profile management.
2. **Station Management:** Station registration, approval workflow, machine tracking.
3. **Booking:** Pre-booking, walk-in, and maintenance slots with lifecycle tracking.
4. **Payments:** Multi-mode payment processing and transaction verification.
5. **Notifications:** Automated alerts triggered by booking and system events.

2 Project Motivation

With the increasing adoption of electric vehicles, charging infrastructure needs to become more reliable, accessible, and digitally manageable. Many charging stations still depend on manual processes, which can lead to overbooking, unclear availability, lack of verification, and poor customer experience.

Electrify aims to create a structured platform where EV users can confidently book charging slots and station owners can manage their infrastructure efficiently.

3 Problem Statement

EV users and station operators face the following core problems:

- Difficulty in finding available nearby charging stations.
- No proper digital system for pre-booking charging slots.
- Lack of real-time machine availability tracking.
- Manual and error-prone station management by station owners.
- No centralized approval system for station registration.
- Poor role separation between customers, owners, managers, and admins.

4 Proposed Solution

Electrify provides a complete role-based EV charging platform. It gives customers a clean dashboard for booking and station discovery, while station owners receive a station management dashboard. Superadmins have a separate control panel for reviewing station requests and managing approvals.

The platform follows a scalable full-stack architecture using React, TypeScript, Node.js, Express, Prisma, MySQL, Cloudinary, and JWT authentication.

5 System Architecture

5.1 Architectural Style

The project follows the **Clean Architecture** pattern, which separates the codebase into concentric layers of concern. This ensures that business logic remains independent of frameworks, databases, and UI technologies, making the system testable and maintainable.

5.1.1 Backend Layers

- **Core (Domain):** Contains the business entities and domain models—`Machine`, `Booking`, `User`—defined without any framework dependencies.
- **Application:** Houses use-case implementations and orchestration logic.
- **Infrastructure:** Integrates with external services—Prisma ORM for MySQL and Cloudinary for file storage.
- **Presentation:** Express REST controllers, route definitions, and request validation middleware.

5.1.2 Frontend Layers

- **Presentation Layer:** React 19 components and pages styled with Tailwind CSS v4.
- **Domain Layer:** Client-side models and business logic specific to the UI.
- **Application Layer:** State management and API interaction using custom hooks.

5.2 Technology Stack

Table 1: Technology Stack Summary

Layer	Technology
Frontend	React 19, TypeScript, Vite, Tailwind CSS v4, Zod, React Router
Backend	Node.js, Express 5, TypeScript, Prisma ORM
Database	MySQL 8.x
Auth	JWT (access + refresh tokens), bcrypt
Storage	Cloudinary (document/image uploads)
Testing	Vitest
Dev Tools	npm, Git, GitHub, Postman, Prisma Studio

5.3 Project Directory Structure

The repository is structured as a monorepo containing independent backend and frontend packages:

Listing 1: Repository Structure

```
SD_Group_Project/
          backend/              # Express API server
                prisma/           #   Prisma schema & migrations
                src/
                    application/  #   Use-cases / business logic
                    config/       #   Environment & app config
                    core/         #   Domain models / interfaces
                    infrastructure/ # DB clients, external services
                    presentation/ #   Routes, controllers, middleware
                    shared/       #   Shared utilities & response helpers
                    app.ts        #   Express app setup
                    server.ts     #   Server entry point
          frontend/             # React + Vite SPA
                src/
```

```
            tests/                  # Test suites
            docs/                   # Documentation
            diagrams/               # UML & use-case diagrams
```

6 Core Features

6.1 User Personas and Role-Based Access

The system defines four distinct user roles, each with a dedicated interface and permission set:

Table 2: User Roles and Responsibilities

Role	Responsibilities
Customer	Search stations, view machine availability, book slots, make payments, receive notifications.
Station Owner (Admin)	Register new charging stations, invite managers, handle station requests.
Manager	Manage day-to-day station operations, handle join requests from customers and staff.
SuperAdmin	Approve station registration requests, manage all users and stations platform-wide.

6.2 Authentication and Security

- **JWT Authentication:** Access tokens (short-lived, default 15 minutes) paired with refresh tokens (7-day rotation) for secure, stateless session management.
- **Password Security:** Passwords are hashed using `bcrypt` before storage.
- **Soft Deletion:** User and station records support `isDeleted` flags to preserve data integrity without permanent removal.
- **Cloudinary Document Uploads:** Government ID verification documents are uploaded securely to Cloudinary and stored as URLs.

6.3 Station and Machine Management

Station owners can register new charging stations, providing geolocation data (latitude/longitude), helpline numbers, address, and operational schedules. Each station contains multiple **Machines**, which track:

- Machine type and machine code.
- License information.
- Minimum fee and fee per hour.
- Active/inactive operational status.

Stations pass through an approval workflow: an owner submits a `StationRequest`, which is reviewed and approved by a SuperAdmin before the station goes live.

6.4 Booking Lifecycle

The booking module supports three modes:

1. **Pre-Book:** Customers reserve a specific time slot in advance.
2. **Walk-In:** On-demand charging without a prior reservation.
3. **Internal Slot:** Reserved for maintenance activities.

Each booking transitions through the following statuses: `PENDING` → `IN_PROGRESS` → `COMPLETED` / `CANCELLED`.

Conflict detection ensures that no two bookings can overlap for the same machine.

6.5 Payments

The payment system supports the following modes:

- UPI
- Credit/Debit Card
- Net Banking
- Cash

Each `Payment` record is linked to a booking and a user, and carries a `PaymentStatus` (`PENDING` or `COMPLETED`) along with the transaction amount.

6.6 Notifications

The notification engine supports three delivery channels:

- **Push:** Real-time in-app alerts.
- **Email:** Booking confirmations, status updates.
- **SMS:** Critical operational alerts.

7 UML Diagrams

7.1 Use Case Diagram

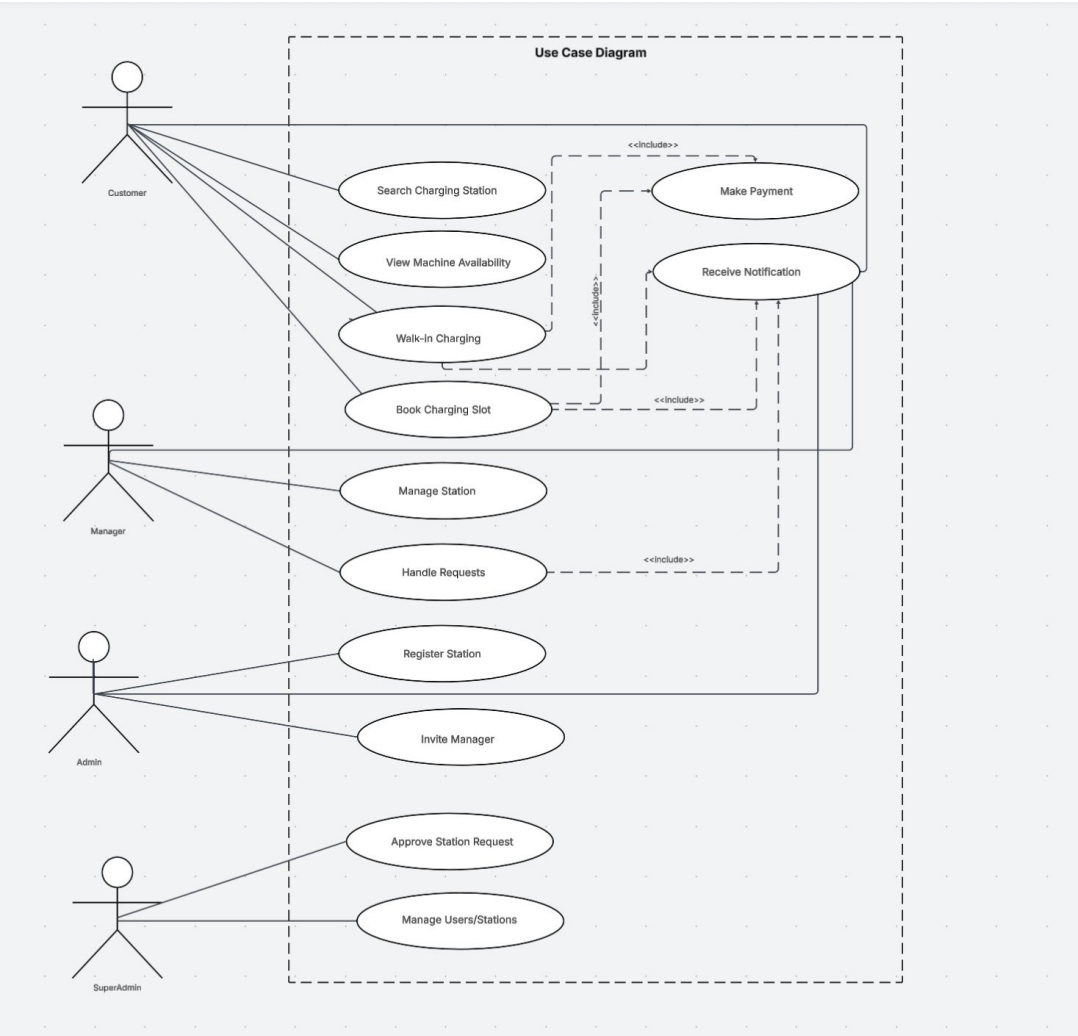

Figure 1: Use Case Diagram – EV Charging System

The use case diagram (Figure 1) illustrates the functional scope of the system across all four actor roles.

7.1.1 Customer Use Cases

- **Search Charging Station:** Browse available stations by location.
- **View Machine Availability:** Check real-time availability of machines at a station.
- **Walk-In Charging:** Initiate an on-demand charging session. *(includes: Make Payment, Receive Notification)*
- **Book Charging Slot:** Reserve a time slot in advance. *(includes: Make Payment, Receive Notification)*
- **Make Payment:** Process payment for a completed or upcoming session.

- **Receive Notification:** Get real-time alerts about booking status and system updates.

7.1.2 Manager Use Cases

- **Manage Station:** Update station details, machines, and operational schedules.
- **Handle Requests:** Review and respond to manager join requests and customer complaints. *(includes: Receive Notification)*

7.1.3 Admin (Station Owner) Use Cases

- **Register Station:** Submit a new station for platform approval.
- **Invite Manager:** Send invite to a user to manage the station.

7.1.4 SuperAdmin Use Cases

- **Approve Station Request:** Review and approve or reject pending station registrations.
- **Manage Users/Stations:** Platform-wide control over all user accounts and station records.

Note: The use case diagram reflects the updated system design including the Manager role and the Handle Requests workflow, which supersedes earlier drafts.

7.2 Sequence Diagram

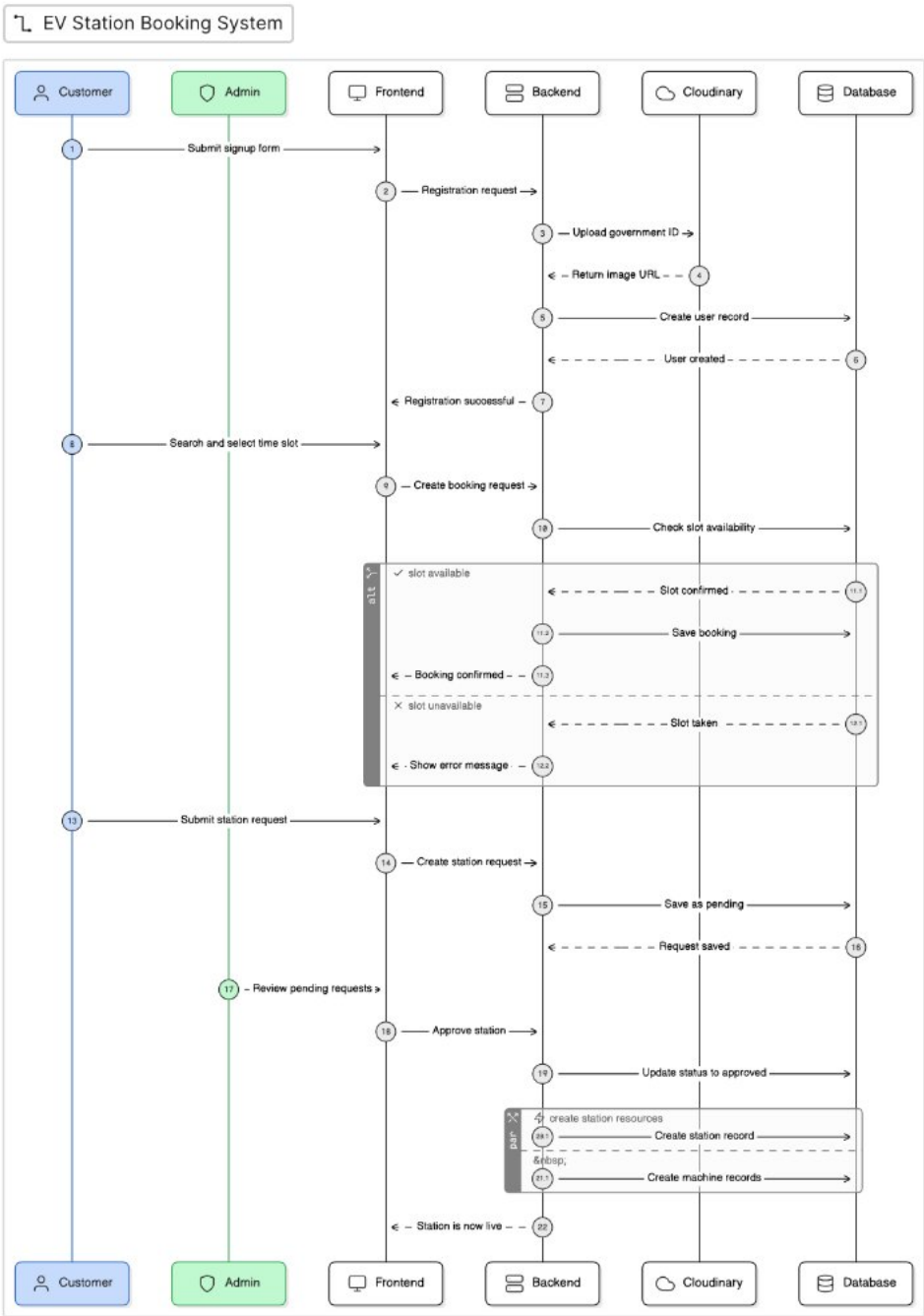

Figure 2: Sequence Diagram – EV Station Booking System

The sequence diagram (Figure 2) models two primary system flows: **Customer Registration & Booking** and **Station Registration & Approval**.

7.2.1 Flow 1: Customer Registration and Slot Booking

1. Customer submits signup form via the Frontend.
2. Frontend sends a registration request to the Backend.

3. Backend uploads the government ID to Cloudinary and receives an image URL in return.
4. Backend creates a user record in the Database.
5. On success, the Frontend displays a registration confirmation to the customer.
6. Customer searches for a station and selects a time slot.
7. Frontend sends a booking creation request to the Backend.
8. Backend queries the Database to check slot availability.
   - *Slot available:* Database confirms availability; Backend saves the booking; customer receives a confirmation.
   - *Slot unavailable:* Database returns a conflict; Frontend shows an error message.

7.2.2 Flow 2: Station Registration and Admin Approval

1. Customer (acting as a station owner) submits a station registration request.
2. Frontend creates a station request via the Backend, which saves it as `PENDING` in the Database.
3. Admin reviews pending requests.
4. Admin approves the station; Backend updates the status to `APPROVED`.
5. Backend creates the station record and associated machine records in the Database.
6. Station is now live and visible to customers.

7.3 UML Class Diagram

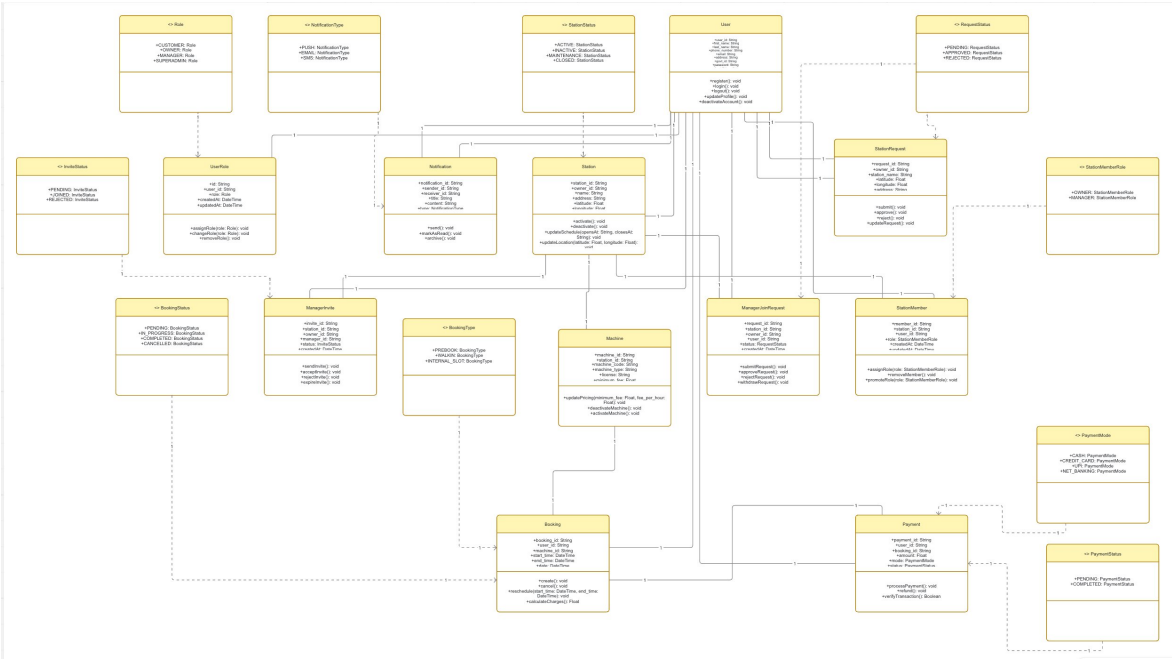

Figure 3: UML Class Diagram – Domain Model

The class diagram (Figure 3) defines the domain model of the system. Key classes and their relationships are described below.

7.3.1 Core Entities

- **User:** Central entity with attributes for credentials, contact info, government ID, and refresh tokens. Methods include `register()`, `login()`, `logout()`, `updateProfile()`, and `deactivateAccount()`.
- **Station:** Represents a charging station with location, schedule, and status. Methods: `activate()`, `deactivate()`, `updateSchedule()`, `updateLocation()`.
- **Machine:** Belongs to a Station. Tracks machine type, fees, and license. Methods: `updatePricing()`, `deactivateMachine()`, `activateMachine()`.
- **Booking:** Links a User and a Machine with a time window. Methods: `create()`, `cancel()`, `reschedule()`, `calculateCharges()`.
- **Payment:** Associated with a Booking. Methods: `processPayment()`, `refund()`, `verifyTransaction()`.
- **Notification:** Sent from one User to another. Methods: `send()`, `markAsRead()`, `archive()`.

7.3.2 Supporting Entities

- **UserRole:** Associates a User with a system Role.
- **StationMember:** Associates a User with a Station in a specific role.

- **ManagerInvite:** Tracks invitations from owners to prospective managers.
- **ManagerJoinRequest:** Tracks join requests from users to become station managers.
- **StationRequest:** Represents a pending station registration awaiting Super-Admin approval.

7.3.3 Enumerations

Table 3: System Enumerations

Enum	Values
Role	CUSTOMER, OWNER, MANAGER, SUPERADMIN
StationStatus	ACTIVE, INACTIVE, MAINTENANCE, CLOSED
BookingStatus	PENDING, IN_PROGRESS, COMPLETED, CANCELLED
BookingType	PREBOOK, WALKIN, INTERNAL_SLOT
PaymentMode	CASH, CREDIT_CARD, UPI, NET_BANKING
PaymentStatus	PENDING, COMPLETED
NotificationType	PUSH, EMAIL, SMS
RequestStatus	PENDING, APPROVED, REJECTED
InviteStatus	PENDING, JOINED, REJECTED
StationMemberRole	OWNER, MANAGER

7.4 Entity–Relationship Diagram

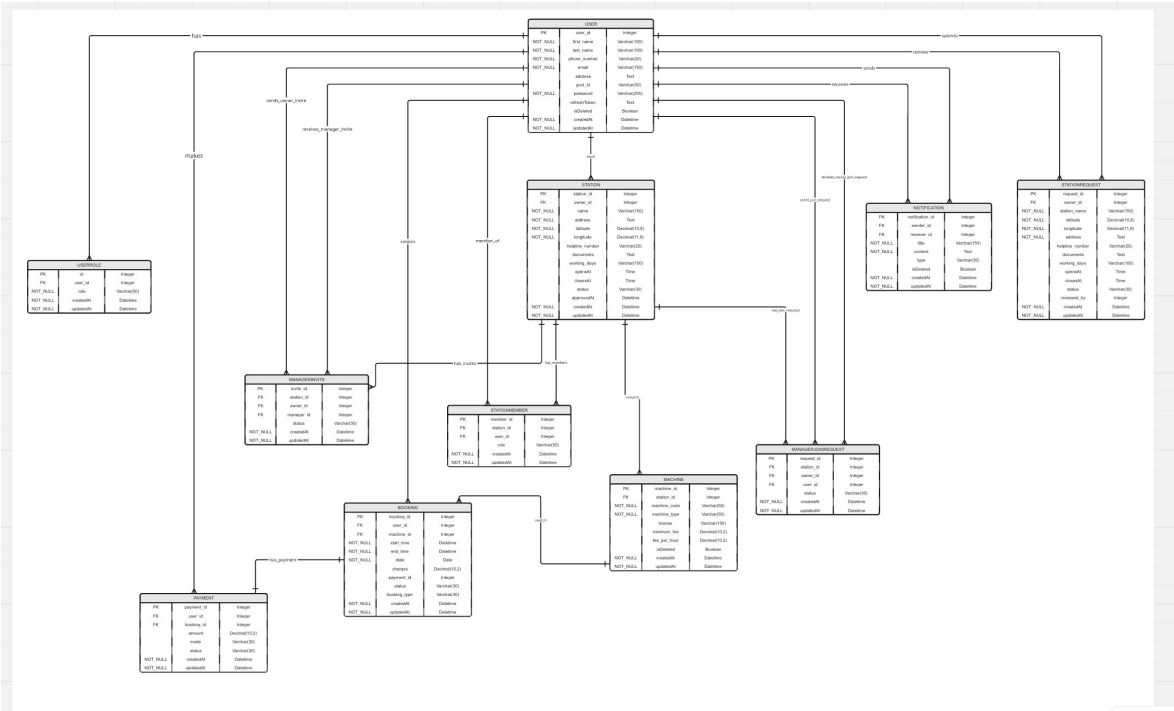

Figure 4: Entity–Relationship Diagram – Database Schema

The ER diagram (Figure 4) maps the physical database schema. Each entity is represented as a table with primary keys (PK), foreign keys (FK), field types, and nullability constraints.

7.4.1 Tables and Key Relationships

Table 4: Database Tables and Their Primary Relationships

Table	Key Relationships
USER	Central table; referenced by nearly all other entities via `user_id`.
USERROLE	FK to USER; stores role per user.
STATION	FK `owner_id` → USER; one owner has many stations.
MACHINE	FK `station_id` → STATION; each station has many machines.
STATIONMEMBER	FKs to STATION and USER; many-to-many station membership.
MANAGERINVITE	FKs to STATION, owner USER, and manager USER.
MANAGERJOINREQUEST	FKs to STATION, owner USER, and requesting USER.
STATIONREQUEST	FK `owner_id` → USER; reviewed by superadmin.
BOOKING	FKs to USER and MACHINE; one machine can have many bookings.
PAYMENT	FKs to USER and BOOKING; one booking has one payment.
NOTIFICATION	FKs to sender USER and receiver USER.

8 System Design Patterns

Electrify uses multiple software design patterns to improve code structure, scalability, and maintainability.

8.1 Singleton Pattern

The Singleton Pattern is used in the database client layer. The backend uses a single shared database client instance instead of creating a new Prisma client on each request. This approach:

- Prevents multiple unnecessary database connections.
- Reduces memory usage and improves performance.
- Manages database connection pooling properly.
- Provides one centralized database access point.

In Electrify, the database client is accessed through a centralized class:

Listing 2: Singleton Database Client Access

```
DatabaseClient.getInstance()
```

8.2 Factory Pattern

The Factory Pattern is used in the payment gateway module. Since the project supports multiple payment modes (UPI, card, net banking, cash), the object creation logic is separated from the main business logic. This allows:

- Creating different payment gateway objects dynamically at runtime.
- Adding new payment methods without modifying existing service logic.
- Keeping payment-related code loosely coupled and testable.
- Making the payment system more scalable for future gateways.

Listing 3: Factory Pattern — Payment Gateway

```
PaymentGatewayFactory.create("UPI")
PaymentGatewayFactory.create("CARD")
```

8.3 Repository Pattern

The Repository Pattern separates database query logic from business logic. Services do not directly write database queries; instead, they interact through repository interfaces. Examples of repositories in the project include:

- `UserRepository`
- `StationRepository`
- `MachineRepository`
- `BookingRepository`
- `PaymentRepository`

This keeps services clean, enables query reuse, simplifies testing via mocked repositories, and allows future database migrations with minimal impact on business logic.

8.4 Layered Architecture Pattern

The complete backend follows a Layered Architecture Pattern where each layer has a single clear responsibility:

- **Controllers** handle HTTP requests and delegate to services.
- **Services** handle business use-case logic.
- **Repositories** handle all database operations.
- **Infrastructure** integrates external tools like Prisma and Cloudinary.
- **Shared utilities** handle responses, errors, and common helpers.

This layered structure makes the system easy to debug, test, and extend independently.

9 SOLID Principles

Electrify is designed to follow the five SOLID principles to keep the codebase clean, maintainable, and extensible.

9.1 Single Responsibility Principle (SRP)

Every class or module has exactly one main responsibility:

- **Controllers** are responsible only for handling HTTP request and response.
- **Services** are responsible only for business logic execution.
- **Repositories** are responsible only for database operations.
- **Middleware** is responsible only for validation, authentication, or authorization.
- **Cloudinary service** is responsible only for file upload handling.

9.2 Open/Closed Principle (OCP)

Code is open for extension but closed for modification. In Electrify, the payment gateway system follows this principle: new payment methods can be added by creating new gateway implementations without changing the existing payment service logic.

9.3 Liskov Substitution Principle (LSP)

Any payment gateway implementation can replace another if it conforms to the same payment interface. High-level services remain unaffected by switching concrete implementations.

9.4 Interface Segregation Principle (ISP)

Focused, specific interfaces are defined for repositories, services, and payment providers rather than one large interface that forces classes to implement unused methods.

9.5 Dependency Inversion Principle (DIP)

High-level modules depend on abstractions rather than concrete implementations:

- Services depend on repository interfaces, not concrete repository classes.
- Controllers depend on service abstractions.
- The payment service depends on payment gateway abstractions.

This improves testability, flexibility, and long-term scalability of the system.

10 API Modules

The backend exposes a REST API organized into five functional module groups. All authenticated routes require a valid JWT passed in the `Authorization: Bearer <token>` header. File-upload routes (such as `/register` and `/station-request`) use **Multer** middleware for multipart/form-data handling.

10.1 Authentication & Account Management — /api/auth

Table 5: Authentication & Account Management Endpoints

Method	Endpoint	Description	Access
POST	/register	Create a new user account with Govt ID upload.	Public
POST	/login	Authenticate user and receive access tokens.	Public
POST	/logout	Invalidate the current user session.	Auth
GET	/me	Get currently logged-in user's details.	Auth
PATCH	/change-password	Update account password.	Auth
PATCH	/activate-account	Re-activate a soft-deleted account.	Public
DELETE	/deactivate-account	Soft-delete the user's own account.	Auth

10.2 User Profile Management — /api/user

Table 6: User Profile Management Endpoints

Method	Endpoint	Description	Access
PATCH	/profile	Update personal profile information.	Auth
PATCH	/govt-id	Update or re-upload the Government ID document.	Auth

10.3 Station Requests — /api/station-request

Used by Owners to register new charging stations and by SuperAdmins to verify them.

Table 7: Station Request Endpoints

Method	Endpoint	Description	Access
POST	/	Submit a new station registration request.	Owner
GET	/	View station requests (filter by user).	Owner/Admin
GET	/:request_id	View details of a specific station request.	Auth
PATCH	/:request_id	Update details of a pending request.	Owner
PATCH	/:request_id/approve	Approve request; system creates a Station record.	SuperAdmin
PATCH	/:request_id/reject	Reject a station request.	SuperAdmin
GET	/status	Filter requests by status (Pending/Approved).	SuperAdmin

10.4 Charging Stations — /api/station

Table 8: Charging Station Endpoints

Method	Endpoint	Description	Access
GET	/	Search and list all available charging stations.	Public/Auth
GET	/:station_id	Get full details of a specific station.	Public
PATCH	/:station_id	Update station details (address, schedule).	Owner
PATCH	/:station_id/status	Change station status (Active/-Maintenance).	Owner/Manager
DELETE	/:station_id	Soft-delete a station.	Owner

10.5 Health & System — /api/health

Table 9: Health Check Endpoint

Method	Endpoint	Description	Access
GET	/api/health	Check if the API server and database are active.	Public

10.6 Authorization Notes

- All authenticated routes require the header: `Authorization: Bearer <access_token>`.
- File-upload routes (`/register`, `/govt-id`, `/station-request`) use **Multer** middleware for `multipart/form-data` handling before reaching the controller.

- Role-based guards enforce access at the route level — a Customer cannot call Owner-only endpoints, and vice versa.
- The access token has a short expiry (`15m`); the refresh token (`7d`) is used to obtain a new access token transparently.

11 Frontend UI Design

The frontend is built with React 19 and TypeScript using Vite as the build tool. It follows a component-driven architecture with reusable UI elements, context providers, and Zod-based form validation.

11.1 Pages

- Login and Signup
- Customer Dashboard (station discovery and booking)
- Station Management Dashboard (for owners)
- Superadmin Approval Panel
- Profile Management

11.2 Key Components

- Sidebar and Navbar for navigation.
- Station cards, vehicle cards, and booking forms.
- Role-specific dashboard components.

11.3 Application Layer

- **Context:** Authentication context for user session management.
- **Services:** API service layer for all backend communication.
- **Validation:** Zod schemas for client-side form validation.

The UI is styled with Tailwind CSS and follows a responsive layout compatible with different screen sizes.

12 Testing

Vitest is the primary testing framework. Testing covers backend services, validation schemas, utility functions, and frontend components.

Planned and implemented testing areas include:

- Authentication service testing (login, signup, token validation).
- Signup and login form validation testing with Zod schemas.
- Protected route authorization testing.

- API response format and status code testing.
- Role-based UI rendering testing on the frontend.

13 Challenges Faced

During development, the team encountered several technical and architectural challenges:

- Managing seamless frontend-backend integration across a monorepo.
- Resolving CORS issues between the Vite dev server and the Express API.
- Designing a correct role-based dashboard routing flow for four distinct personas.
- Implementing JWT authentication with access and refresh token rotation correctly.
- Handling Cloudinary file uploads with proper error handling and URL storage.
- Keeping request and response types fully consistent across TypeScript boundaries.
- Preventing field name mismatches between frontend form data and backend DTOs.

14 Future Scope

Electrify has a strong foundation for future expansion. Planned enhancements include:

- **Real-time station availability** using WebSockets for live occupancy updates.
- **Map-based station search** with distance-based nearest station recommendations.
- **Online payment gateway integration** (Razorpay, Stripe).
- **QR-based station check-in** for walk-in customers.
- **Mobile application** for Android and iOS customers.
- **Analytics dashboard** for station owners with revenue and booking reports.
- **AI-based charging demand prediction** for dynamic slot allocation.
- **Full notification system** via email, SMS, and push alerts.

15 Environment Configuration

15.1 Required Environment Variables

Table 10: Required Backend Environment Variables

Variable	Description
DATABASE_URL	MySQL connection string used by Prisma.
JWT_SECRET	Secret key for signing JWT tokens.
ACCESS_TOKEN_EXPIRY	Lifespan of access tokens (e.g., 15m).
REFRESH_TOKEN_EXPIRY	Lifespan of refresh tokens (e.g., 7d).
CLOUDINARY_CLOUD_NAME	Cloudinary cloud identifier.
CLOUDINARY_API_KEY	Cloudinary API key.
CLOUDINARY_API_SECRET	Cloudinary API secret.

15.2 Optional Environment Variables

Table 11: Optional Backend Environment Variables

Variable	Description	Default
PORT	Port the server listens on.	8000
NODE_ENV	Runtime environment.	development
FRONTEND_URL	Allowed CORS origin.	http://localhost:5173

16 Setup and Deployment

16.1 Prerequisites

- Node.js $\geq$ 18
- npm $\geq$ 9
- MySQL 8.x (local installation or managed cloud instance)
- A Cloudinary account for document storage

16.2 Local Development Setup

1. **Clone the Repository:**

```
git clone <repository-url>
cd SD_Group_Project
```

2. **Create the Database:**

```
CREATE DATABASE ev_charging;
```

3. **Backend Setup:**

```
cd backend
npm install
```

```
cp .env.example .env   # configure variables
npx prisma generate
npx prisma db push
npm run dev
```

4. **Frontend Setup:**

```
cd frontend
npm install
npm run dev
```

The backend API will be available at `http://localhost:8000` and the frontend at `http://localhost:5173`.

16.3 Available Scripts

Table 12: Backend npm Scripts

Command	Description
`npm run dev`	Start development server with hot reload.
`npm run build`	Compile TypeScript to `dist/`.
`npm start`	Run the production build.
`npm test`	Run tests with Vitest.
`npm run test:watch`	Run tests in watch mode.
`npm run type-check`	Type-check without emitting output.

Table 13: Frontend npm Scripts

Command	Description
`npm run dev`	Start Vite development server.
`npm run build`	Type-check and build for production.
`npm run lint`	Run ESLint.
`npm run preview`	Preview the production build.

17 Conclusion

Electrify is a practical and scalable EV charging station management system. It solves the problem of EV charging accessibility by providing booking, station management, role-based dashboards, secure authentication, and admin approval features.

The project uses modern full-stack technologies and follows clean architecture principles, design patterns (Singleton, Factory, Repository, Layered Architecture), and all five SOLID principles, making it maintainable, scalable, and suitable for real-world expansion.

Key design decisions—JWT-based authentication with token rotation, role-based access control across four user personas, conflict-aware booking management, the Repository and Factory patterns for decoupled data and payment handling—collectively

make Electrify a production-ready foundation for the next generation of EV charging infrastructure.